

1. Prvočísla

Prvočíslo je takové číslo, které je beze zbytku dělitelné jen samo sebou a číslem 1.

Vytvořte program, který se snaží získat od uživatele prvočíslo.

- Program běží, dokud uživatel nezadá prvočíslo nebo neukončí program.
 - Jakmile zadá prvočíslo, vypíše ho do konzole.
1. Zkuste naimplementovat program s vlastními výjimkami:
 - a. Pokud nezadá číslo, nechte vyvolat výjimku.
 - b. Pokud zadá sudé číslo ($\neq 2$), vyvolejte vhodnou výjimku.
 - c. Pokud zadané číslo není prvočíslo, vyvolejte vhodnou výjimku (s jinou zprávou).
 2. Naimplementujte další variantu programu jen s jednou výjimkou:
 - a. Pokud nezadá číslo, zachyťte vyvolanou výjimku.
 - b. Pokud zadá sudé číslo ($\neq 2$), vypíše do konzole vhodnou hlášku a vraťte *False*.
 - c. Pokud zadané číslo není prvočíslo, vraťte *False*.
 3. Naimplementuje poslední variantu programu bez výjimek:
 - a. Kontrolujte, že zadaný text lze převést na číslo – obsahuje jen číslice.
 - b. Zbytek implementace je analogický k bodu 2. Zamezujte zbytečnému vyvolávání výjimek.

2. Iterátor

Iterátor je programovací paradigma, které udává efektivní způsob procházení prvků kolekce dat.

1. Vytvořte třídu, která se chová jako iterátor nad seznamem:

1. Vytvořte třídu, která přebírá v konstruktoru seznam řetězců jako atribut.
2. Přidejte třídě atribut *index*, který bude sloužit jako ukazatel na zrovna vrácený prvek.
3. Přidejte třídě metodu *má_další_prvek*, která vrátí pravdivostní hodnotu, zda jsou v seznamu ještě další nevypsane prvky.
4. Přidejte třídě metodu *další_prvek*, která vrátí řetězec na současném *indexu* a následně *index* posune o 1.
5. Přidejte třídě metodu *reset*, která posune *index* tak, aby bylo možné projít seznamem znovu od začátku.
6. Vyzkoušejte implementaci:
 - a. Vytvořte instanci této třídy s nějakým seznamem.
 - b. Pomocí *while* cyklu a metod instance vypište všechny prvky seznamu.
 - c. Resetujte stav iterátoru a vypište obsah znovu.

2. Výjimková implementace:

1. Vytvořte třídu, která dědí od třídy z předchozí implementace.
2. Překryjte metodu *další_prvek* tak, aby sama uvnitř kontrolovala, zda má ještě k dispozici další prvek. Pokud ano, vrátí ho stejným způsobem. Pokud ne, vyvolá výjimku *IndexError*.
3. Vyzkoušejte implementaci:
 - a. Vytvořte *try-except* blok, který odchytává správně výjimky.
 - b. Vypíšte obsah seznamu pomocí *while* cyklu.
 - c. Resetujte stav iterátoru a vypište obsah znovu.

3. Zabudovaná implementace:

1. Vytvořte třídu, která má stejný konstruktor jako předchozí třídy.
2. Naimplementujte třídě zabudovanou metodu *__iter__*, která resetuje stav iterátoru a vrátí sama sebe.
3. Naimplementujte třídě zabudovanou metodu *__next__*, která zkontroluje, zda má k dispozici další prvek a pokud ano, vrátí ho a posune svůj ukazatel. Pokud ne, vyvolá výjimku *StopIteration*.
4. Vyzkoušejte implementaci:
 - a. Vypíšte obsah seznamu pomocí *try-except* a *while* cyklu.
 - b. Vypíšte obsah seznamu pomocí *foreach* cyklu (*for prvek in instance*).
 - c. Vypíšte obsah seznamu pomocí „seznamové lambdy“ (*[a for a in instance]*). A ještě jednou bez manuálního resetování.

4. Náhodná implementace:

1. Vytvořte podobným způsobem jako v bodě 3 iterátor, který:
 - a. Přebírá v konstruktoru jen hodnotu *šance* jako atribut (reálné číslo mezi 0 a 1).
 - b. Implementuje zabudovanou metodu `__iter__` – vrátí sama sebe.
 - c. Implementuje zabudovanou metodu `__next__`:
 - i. Při každém volání vygeneruje pomocí knihovny *random* číslo mezi 0 a 1.
 - ii. Pokud je *šance* nižší než vygenerované číslo, ukončí iterátor.
 - iii. Pokud není, vrátí číslo 1.
2. Vyzkoušejte implementaci:
 - a. Vypište obsah seznamu pomocí „seznamové lambdy“ třikrát po sobě.